

Parallel AI Model Training and Hyperparameter Search

Next-Day Stock Movement Prediction with PySpark MLlib

Module	MDS3 Parallel Computing	Date	21 July 2026
Name	Yalin Luo		
Environment	Google Colab, local[*]	Resources	2 CPU cores, 8 partitions

Executive Summary

A Spark ML workflow was developed to predict whether a stock would close higher on the next trading day. The experiment compared four Random Forest configurations and benchmarked TrainValidationSplit with parallelism levels 1, 2, and 4. Across two counter-ordered runs, median tuning time fell from 36.58 s at parallelism=1 to 31.80 s at parallelism=2 (1.15x speedup). Parallelism=4 produced no further benefit (31.92 s), consistent with the two-core runtime. Configuration E2 (20 trees, depth 10) was selected. It improved test F1 from 0.4392 to 0.4700, although the absolute predictive performance remained limited.

1. Concept Check

Q1. What is a hyperparameter?

A. A user-defined setting chosen before training, such as tree count or maximum depth.

Q2. Parameter vs. hyperparameter

A. Model parameters are learned from data; hyperparameters control how the model is trained.

Q3. Why use a baseline?

A. It provides a fixed reference for measuring gains in quality and changes in runtime.

Q4. Hyperparameter search

A. Training and evaluating multiple candidate settings to identify an effective configuration.

Q5. Purpose of a parameter grid

A. It enumerates reproducible combinations of candidate hyperparameter values.

Q6. Train-validation split

A. Training data are divided once into fitting and validation subsets for model selection.

Q7. Parallelism in tuning

A. Several candidate configurations can be evaluated concurrently.

Q8. Data vs. experiment parallelism

A. Data parallelism splits one workload across partitions; experiment parallelism runs different models concurrently.

Q9. Why speedup is not linear

A. Scheduling, memory pressure, task coordination, unequal workloads, and limited CPU cores reduce scaling efficiency.

2. Dataset and Prediction Problem

The experiment used a historical stock-price CSV and formulated next-day movement as a binary classification task. The label equals 1 when the following trading day's closing price is higher than the current close, and 0 otherwise. After feature engineering, 26,000 usable observations remained: 21,000 training rows and 5,000 external test rows.

Feature	Meaning
daily_return	One-day percentage price return
return_5d	Five-day price return
ma_ratio_5	Close relative to the five-day moving average
ma_ratio_20	Close relative to the twenty-day moving average
volatility_10d	Ten-day standard deviation of daily returns
volume_change	Daily percentage change in trading volume
price_range	Intraday high-low range relative to close

3. Spark ML Workflow

CSV data	Cleaning	Window features	Time-based split	Encoding + vector	Random Forest + TVS	F1 + runtime comparison
----------	----------	-----------------	------------------	-------------------	---------------------	-------------------------

Spark Window operations created lagged returns, moving averages, volatility, volume change, and the next-day label separately for each ticker. Data were repartitioned into eight partitions and cached. The outer train/test division respected time order. StringIndexer and OneHotEncoder represented the ticker, while VectorAssembler produced the features vector required by Spark ML.

4. Experimental Design

Item	Setting
Algorithm	RandomForestClassifier
Primary evaluator	F1-score
Baseline	numTrees=20, maxDepth=5, seed=42
Tuning grid	numTrees in {20, 50}; maxDepth in {5, 10}
Configurations	E1=(20,5), E2=(20,10), E3=(50,5), E4=(50,10)
Tuner	TrainValidationSplit, trainRatio=0.8, seed=42
Parallelism levels	1, 2, and 4
Benchmark protocol	Warm-up; two runs; reversed order in round 2; median runtime reported

5. Baseline

The fixed baseline used 20 trees and maximum depth 5. Its external test F1-score was 0.4392. All tuning experiments used the same processed training data, evaluator, seed, and parameter grid; only the tuning parallelism changed.

6. Results

Parallelism	Runs	Median (s)	Min (s)	Max (s)	Speedup	Interpretation
1	2	36.58	29.96	43.21	1.00x	Sequential reference
2	2	31.80	29.52	34.09	1.15x	Fastest median
4	2	31.92	31.51	32.33	1.15x	No gain over p=2

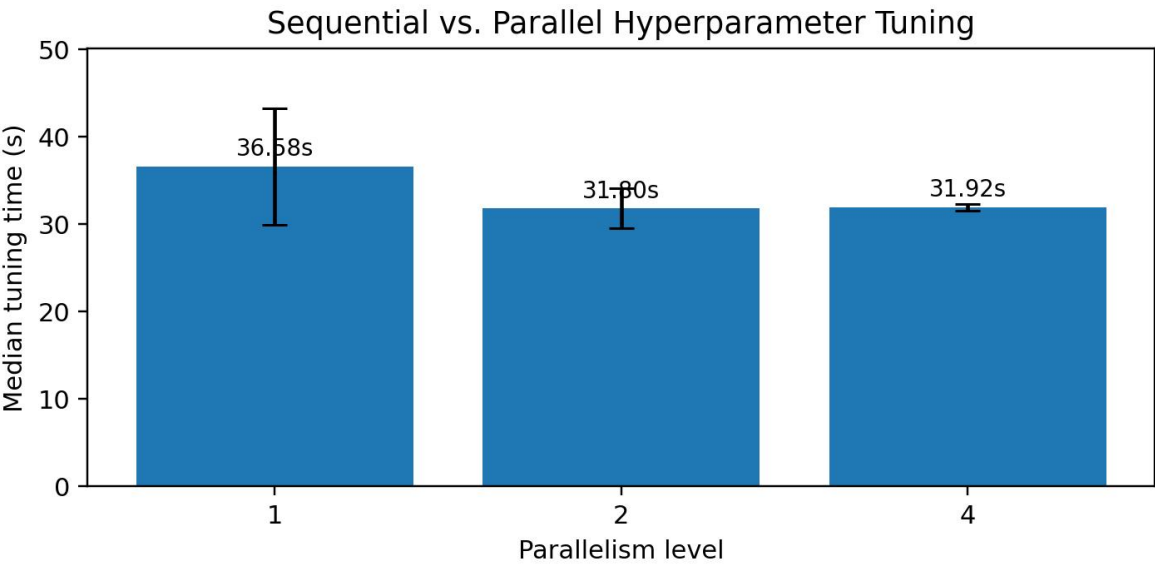


Figure 1. Median tuning time; error bars show the two-run minimum and maximum.

6.1 Hyperparameter Performance

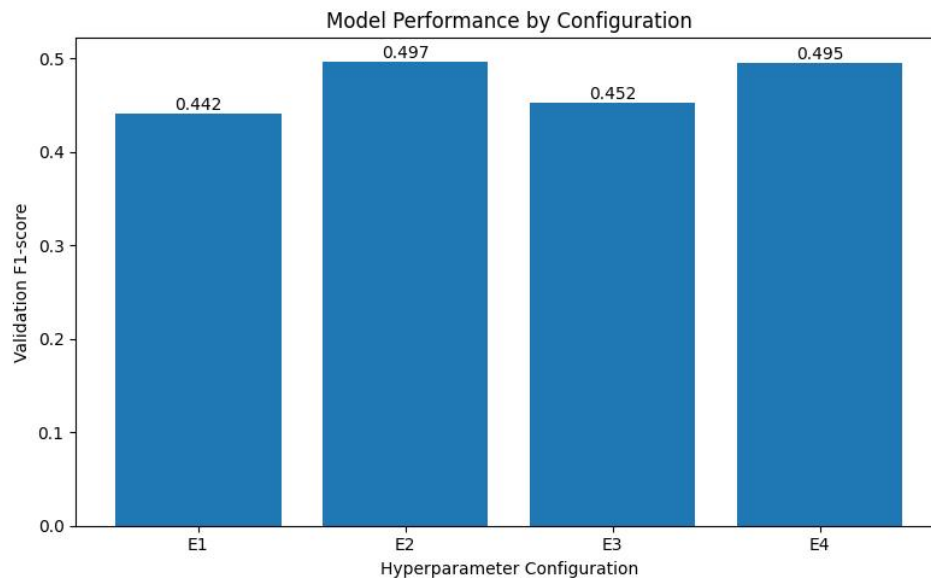


Figure 2 Model Performance by Configuration

ID	numTrees	maxDepth	Validation F1	Cost observation
E1	20	5	0.4417	Lowest complexity; weak validation F1
E2	20	10	0.4972	Best F1; selected
E3	50	5	0.4522	More trees did not offset shallow depth
E4	50	10	0.4954	Near-best F1 but higher cost

A separate time-based cost check also selected E2: it achieved validation F1=0.4892 with a total configuration time of 9.55 s, whereas E4 required 16.55 s and produced F1=0.4730. The exact validation scores differed because the two procedures used different validation partitions, but both identified E2 as the strongest configuration.

6.2 Final Model Selection

Model	Trees / depth	Test F1	Result
Baseline	20 / 5	0.4392	Reference
Selected E2	20 / 10	0.4700	+0.0308 absolute; +7.0% relative

E2 was selected because it produced the highest validation F1 and avoided the additional tree count of E4. The final test improvement was measurable but modest. A test F1 of 0.4700 does not establish practical trading value; it mainly demonstrates the complete Spark model-selection workflow.

7. Discussion and Scalability

Parallel evaluation reduced median tuning time by approximately 13% at parallelism=2. Increasing parallelism to 4 did not improve the median because the Colab runtime exposed only two CPU cores. The additional concurrent work therefore competed for the same CPU and memory resources. The first parallelism=1 run was also substantially slower than its second run, illustrating JVM warm-up, cache effects, and runtime variability. Reporting medians after a warm-up and reversing the execution order produced a more defensible result than relying on a single measurement.

For 1,000 configurations, exhaustive grid search would be impractical in this environment. A larger Spark cluster, random search, staged coarse-to-fine tuning, or Bayesian optimisation would reduce wasted computation. Data should remain cached, and parallelism should be matched to executor cores and available memory rather than increased without limit.

8. Reflection Questions

#	Question	Response
1	Most important concept	Fair comparison: identical data, grid, seed, and evaluator, changing only parallelism.
2	Data vs. experiment parallelism	Partitions parallelise one workload; tuning parallelism evaluates independent configurations concurrently.
3	Why create a baseline?	To quantify whether tuning improved prediction quality and to provide a fixed engineering reference.

#	Question	Response
4	Hyperparameters tested	numTrees controls ensemble size; maxDepth controls tree complexity and overfitting risk.
5	Effect of increased parallelism	Median time fell from 36.58 s to 31.80 s at p=2; p=4 was essentially unchanged at 31.92 s.
6	Linear speedup?	No. Two CPU cores, scheduling overhead, memory contention, and unequal model durations limited scaling.
7	Selected configuration	E2: 20 trees, depth 10, because it combined the strongest F1 with lower cost than E4.
8	Highest accuracy vs. lowest cost	No. The least expensive configurations were weaker; the selected model balanced both objectives.
9	If there were 1,000 configurations	Use a larger cluster and more efficient search rather than exhaustive grid search.
10	Main limitation	Next-day stock movement is noisy; the local two-core benchmark and single-split tuning limit generalisability.

9. Limitations

- Predictive performance remained close to chance; no claim of profitable trading performance is made.
- TrainValidationSplit is not a dedicated walk-forward time-series validation procedure.
- Only two benchmark repetitions were collected on a variable shared Colab runtime.
- The parameter grid was deliberately small to keep the six-hour assignment computationally feasible.

References

- [1] Apache Spark. ML Tuning: Model Selection and Hyperparameter Tuning.
- [2] Apache Spark. PySpark TrainValidationSplit API Documentation.
- [3] Apache Spark. RDD Programming Guide.
- [4] Meng, X. et al. (2016). MLlib: Machine Learning in Apache Spark. Journal of Machine Learning Research, 17(34), 1-7.